RAG Implementation using EHR Knowledge Graph

Chris Hamling

University of Texas at Austin, christopher.hamling@utexas.edu

Powerpoint Slides (Open in PPT to view demo)

Video Presentation

Code Repository

Data Source

1 INTRODUCTION

With the complexity and volume of data in the healthcare industry, organizations have failed to keep up with the cutting edge of technology when it comes to ML/AI applications. In an environment where proper insights can save billions of dollars or even a life, it has been estimated that 97% of data is unstructured, and as much as 80% of that data goes unused entirely. While cutting edge breakthroughs in AI are expected to revolutionize the industry like many others, this barrier to entry has significantly slowed the adoption of such technologies. In this paper we outline the potential for graph retrieval augmented generation (GraphRAG) to enable powerful insights from EHR data with simple text query input. While retrieval augmented generation frameworks are becoming increasingly leveraged for LLM functionality, the limitations lie in the fact that the response is only as good as the underlying document retrieval, generally vector search or semantic search with a few document snippets provided as context. With the combination of knowledge graph technology and LLMs, we can not only answer specific questions found in the text but also allow the LLM to reason about the entire dataset, identifying overarching patterns and aggregating across thousands of datapoints simultaneously.

To approach the problem of unstructured and disorganized healthcare data, we will focus on a small subset of synthetic EHR data generated by Synthea. This data, provided in the form of multiple tabular files with multiple reference keys between them, will be loaded into a pre-configured knowledge base using Neo4j, one of the more mature graph database solutions available today. With this graph database loaded with EHR data, we can then query data in real time to provide relevant context to an LLM for data retrieval, question answering, and summarization.

1.1 Related work

The first noted proposal of GraphRAG technology was published in the Microsoft Research Blog [1] in February 2024 as a method to reason across far more text passages than the standard LLM context window would allow. In this study they demonstrate the improved performance of GraphRAG over baseline RAG in summarizing in cases with a high number of relevant documents as well as whole-dataset reasoning like data themes. They followed this up with a published paper in April 2024 that proposes the framework as a method to enable RAG systems to scale over large quantities of data while handling vague or generalized questions. Their implementation differs in that their knowledge graph is primarily a representation of freeform text data [2]. Within the healthcare space and specifically EHR data from the MIMIC III dataset, another paper outlines the potential for representing semantic relationships between datapoints using a knowledge graph implementation similar to that used in this experiment using Ontotext GraphDB[3]. The primary difference in their approach is the generous use of nodes to represent each individual data attribute, while the approach in this experiment builds nodes for the primary related entities, keeping additional data stored as node properties.

2 METHODOLOGY

2.1 Overall Approach Framework

The functional framework for the GraphRAG implementation in this experiment consists of three distinct building blocks, the graph database itself, an LLM call to generate a graph database query and retrieve contextual information, and an additional LLM call to provide a context-driven response to the initial user input. This approach is built on baseline RAG implementations, while replacing traditional document retrieval with a graph query allowing for access to broader contextual data. In this experiment we replaced vectorized semantic search retrieval with an LLM-generated Cypher query. To orchestrate this multi-stage LLM call, the project uses langchain which includes built-in functionality to interface with Neo4j.

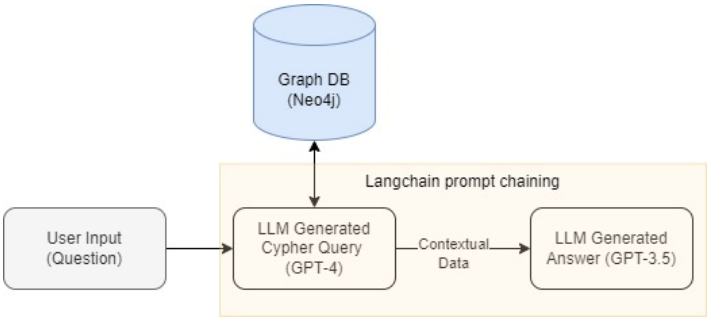

Figure 1: Implemented solution framework.

2.2 Knowledge Graph Implementation

To implement the knowledge graph, Neo4j was used due to its' abundant documentation and project maturity, with a local build of Neo4j Desktop running the DB instance. The original data schema was built in their managed service AuraDB and exported as an import script in Cypher query language, the standard for interacting with a Neo4j DB. Running this script against the local instance imported data directly from the csv files into the graph as nodes representing the key entities (patients, providers, encounters, etc.) and their pre-defined relationship structure. As an expansion on the existing relationships provided in the data, a graph KNN algorithm was then applied to the appropriate entities to add additional semantic connections between similar providers, patients, and other relevant nodes.

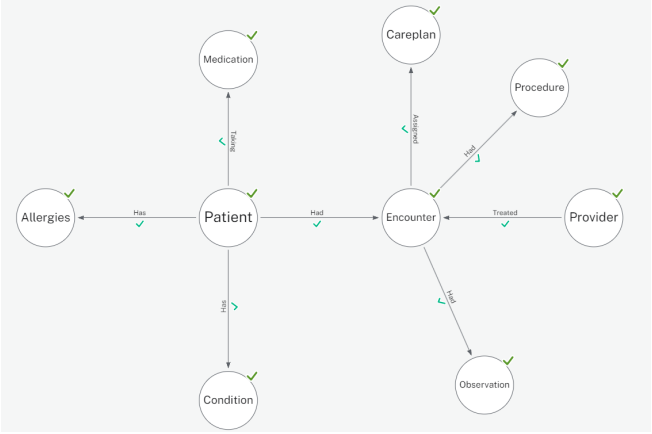

Figure 2: Database schema used to represent EHR data and interactions.

2.3 Query Generation using LLM

With the expansive information encoded into the graph, the next step is to reliably and effectively retrieve that information to provide relevant context to the question answering LLM. Knowledge of how to translate a question into an appropriate Cypher query is not a trivial task for any LLM's and in this experiment the only effective model identified was OpenAI's GPT-4. With our generated database queries extracting all relevant contextual data as a structured JSON response, we can then leverage that context to enable the next step in the model chain and answer the original user question.

2.4 Response Generation using LLM

The expected output of a QA chain is a concise textual response to the users input question, so in the final step we make an additional call to GPT-3.5 which can handle the JSON context and providing a consistent context-driven response. While the context passed to the LLM can often contain additional noise from the generated Cypher query, the model proved effective at reasoning about the provided data and generating an accurate response.

3 RESULTS

Through testing across various tasks including summarization, entity similarity, and aggregate reasoning, the solution excelled in providing concise and accurate responses to nearly all provided user questions. While the system managed to answer most questions with ease in seconds, the primary point of failure was in the generation of Cypher queries, where in some cases the query didn't quite capture the essence of the prompt. An additional point of inaccuracy was identified in the QA model's ability to reason about the provided JSON context, in some example cases taking the last item in a sorted list as the correct answer rather than the first.

4 CONCLUSION

4.1 Findings

Combining the functionality of knowledge graph queries to navigate both broad and deep data sources with the ability of LLMs to reason about text data provides a framework capable of handling a much wider variety of data-driven questions. In the hands of frontline healthcare professionals, insurance companies, or even individual patients, this tool could provide a level of data insight and availability never seen in the industry, without the need for extensive domain or technical expertise. While the experiment strived to prove the potential for knowledge graph storage of EHR data to improve the effectiveness of AI/ML applications, the proof-of-concept also highlights the need for improvement in data availability across the healthcare industry.

4.2 Future Work

This initial implementation provides a foundational framework for Graph-based retrieval augmented generation, however the true value of graph database technology lies in the capability of expansion. While this approach primarily leveraged tabular EHR data, the abundance of visual data, freeform text, and other unstructured data sources in healthcare could be ingested for multi-modal reasoning across the vast ecosystem of healthcare information. Just as we implemented a nearest neighbor algorithm to identify similar patients, there are a vast array of Graph ML algorithms suited to enable prediction, anomaly and cluster detection, and many more complex tasks that can be run at the time of ingestion. The ability to iteratively enhance the database itself without rebuilding from the ground up would be a dramatic improvement over the current state of healthcare data.

REFERENCES

[1] Jonathan Larson and Steven Truitt. 2024. Graphrag: A new approach for discovery using complex information. (April 2024). Retrieved April 21, 2024 from https://www.microsoft.com/en-us/research/blog/graphrag-unlocking-llm-discovery-on-narrative-private-data/

[2] Darren Edge et al. 2024. From local to global: A graph RAG approach to query-focused summarization. (April 2024). Retrieved April 26, 2024 from https://arxiv.org/abs/2404.16130

[3] Bader Aldughayfiq, Farzeen Ashfaq, N.Z. Jhanjhi, and Mamoona Humayun. 2023. Capturing semantic relationships in electronic health records using knowledge graphs: An implementation using MIMIC III dataset and graphdb. (June 2023). Retrieved April 26, 2024 from https://www.mdpi.com/2227-9032/11/12/1762